

LISTA DE EXERCÍCIOS 2.4

1. Forneça três exemplos no qual *multithreading* oferece melhor desempenho do que uma solução baseada em *thread* única.

Resposta: (1) Um servidor web que responde cada requisição em uma *thread* separada; (2) Uma aplicação paralelizada como por exemplo, para a multiplicação de matrizes, onde diferentes padrões de matrizes podem ser trabalhados em paralelo; (3) Um programa de GUI interativa como no caso de um debugger, onde uma *thread* é usada para monitorar a entrada do usuário, outra *thread* é usada para representar a aplicação em execução, e uma terceira para monitorar o desempenho.

2. Quais são as diferenças entre *threads* de nível de usuário e *threads* de nível de kernel? Sob quais circunstâncias um tipo é melhor do que o outro?

Resposta: *threads* de nível de usuário são desconhecidas pelo kernel, enquanto as de nível de kernel, são conhecidas. Em sistemas com mapeamento M:1 ou M:N, *threads* de usuário são escalonadas pela biblioteca de *thread* e o kernel escalona *threads* de kernel. *Threads* de kernel, apesar de estarem associadas com um processo, podem executar independentemente em núcleos separados (i.e. paralelismo real) enquanto cada *thread* de usuário pertencem a um processo e geralmente é delegada a um processador, executando durante o tempo delegado àquele processo. *Threads* de kernel são geralmente mais “caras” de manter do que *threads* de usuário, considerando que estas devem ser representadas com uma estrutura de dados em nível de kernel. Finalmente, em caso de chamadas bloqueantes para dispositivos de E/S, em uma implementação baseada em *threads* de usuário, o processo é colocado em uma fila de espera, cedendo sua vez, mesmo havendo ainda tempo de execução restante, enquanto que em uma implementação de *thread* de kernel, caso haja *threads* em condições de executar, o processo continua executando até o limite de seu tempo de execução na CPU.

3. Descreva as ações tomadas pelo *kernel* para realizar o chaveamento de contexto entre *threads* em nível de *kernel*.

Resposta: O chaveamento de contexto entre *threads* de kernel tipicamente requer salvar o valor dos registradores de CPU da *thread* sendo interrompida (*switched out*) e restaurar o conteúdo dos registradores da CPU a partir do conteúdo da nova *thread* sendo escalonada.

4. Quais recursos são usados quando uma *thread* é criada? Como eles são diferentes daqueles usados quando um processo é criado?

Resposta: Considerando que uma *thread* é interna a um processo, a criação de *thread* usa tipicamente menos recursos do que a criação de um processo. Dessa forma, parte-se do arcabouço criado no momento da criação do processo, o qual inclui um bloco de controle de processo (Process Control Block ou PCB), que é uma estrutura de dados relativamente grande. O PCB inclui um mapa de memória, a lista de arquivos abertos e variáveis de ambiente. Alocar e gerenciar o mapa de memória é tipicamente a atividade mais custosa em relação a consumo de tempo. Considerando a atividade de criação de *thread* (partindo do pressuposto que o processo já existe), criar uma *thread* de espaço de usuário ou de espaço de kernel envolve a alocação de uma estrutura de dados menor para manter um conjunto de registradores (*register set*), pilha (*stack*) e prioridade específicos de cada *thread*.

5. Considere os modelos de *threads* muitos-para-um (M:1), um-para-um (1:1) e muitos-para-

muitos (M:N). Considerando os tipos de *threads* vistos em classe (*threads* de espaço de usuário e *threads* de espaço de kernel), como você situaria cada um dos modelos?

Resposta: considerando *threads* de espaço de usuário, o SO desconhece a existência de *threads* (i.e. não há implementação de *threads* no kernel). Assim sendo, os modelos muitos-para-um ou um-para-um poderiam ser situados como implementações possíveis para *threads* de espaço de usuário. A ressalva se faz ao modelo um-para-um aplicado a *threads* de espaço de usuário, considerando-se que neste modelo o kernel continua desconhecendo a existência de *threads* em espaço de usuário, e que o programa, mesmo dispondo de uma biblioteca de *threads* implementaria apenas uma única *thread*, ou teríamos ainda, um programa sequencial tradicional com uma única *thread* representada em espaço de kernel (i.e. um-para-um). No caso de *threads* de espaço de kernel, os modelos um-para-um e muitos-para-muitos são mais comumente aplicados e implicam em um conhecimento pleno do kernel com relação a existência de *threads*. Em ambos os casos o escalonador de processos do kernel, inclusive, realiza o escalonamento das *threads* criadas pelo programa de usuário. O modelo 1:1 implica no mapeamento de toda *thread* de espaço de usuário em uma *thread* em espaço de kernel. O modelo M:N implica no conceito de um pool de *threads* de kernel, no qual existe uma quantidade limitada de *threads* de kernel criadas que teriam condição de atender a demanda do usuário (em tempo considerado aceitável).

6. Como o modelo em dois níveis se posiciona considerando *threads* de espaço de usuário e *threads* de espaço de kernel? Como este se diferencia, por exemplo, do modelo M:N tradicional?

Resposta: o modelo em dois níveis, por ser uma variação do modelo M:N, garante que o kernel tem conhecimento de *threads*. Logo, seria aplicado a *threads* de espaço de kernel. O modelo em dois níveis acrescenta a possibilidade de atrelar uma *thread* de kernel a uma *thread* de usuário mediante uma prioridade (ex. no caso de uma *thread* de tempo real). Com isso, é possível evitar a sobrecarga de busca por uma nova *thread* de kernel a cada chaveamento de uma *thread* de usuário (e potencialmente o tempo de espera caso não exista uma *thread* de kernel disponível).